

Assignment 03: Robustness Report

Trustworthy Machine Learning – SS 2025

1. Introduction

In this assignment, we developed a robust image classifier to defend against adversarial attacks, specifically **FGSM** and **PGD**. The model was evaluated on clean and adversarial data, and our final approach achieved:

- **Clean Accuracy:** 51.33%
- **FGSM Accuracy:** 39.90%
- **PGD Accuracy:** 38.07%

We present a detailed breakdown of our successful approach, failed experiments, and the most impactful code components throughout our development process.

Repository Contents

- `adversarial_training.ipynb` — Main training notebook
 - `robust_model.pt` — Final model checkpoint
- Link provided for .pt: <https://drive.google.com/drive/folders/1a5Zi9YSve0wRAJx-UpV1Rk1mHdnIAN9n>
-

2. Strategy and Challenges

2.1 Threat Model

Our model was evaluated against:

- Clean samples
- **FGSM** (Fast Gradient Sign Method): single-step attack
- **PGD** (Projected Gradient Descent): multi-step attack

These adversarial inputs aim to mislead the model with imperceptible perturbations. Our strategy involved incorporating adversarial examples directly into the training loop.

2.2 Data Handling

We used the provided dataset `Train.pt`, loaded via a custom `TaskDataset` class. To ensure all inputs were valid, we included:

python

```
if img.mode != "RGB":  
    img = img.convert("RGB")
```

Transformation:

python

```
transform = transforms.Compose([  
    transforms.Resize((32, 32)),  
    transforms.ToTensor()  
])
```

3. Working Robust Training Strategy

3.1 Model

We used `ResNet-18` from `torchvision`, adapted for 10-class classification:

python

```
model = models.resnet18(weights=None)  
model.fc = nn.Linear(model.fc.in_features, 10)
```

This model complies with the competition's allowed model list and has an ideal capacity for 32×32 inputs.

3.2 Adversarial Training

We implemented **FGSM** and **PGD** as part of each training batch:

FGSM:

python

```
def fgsm_attack(model, images, labels, epsilon):  
    images.requires_grad = True  
    outputs = model(images)  
    loss = nn.CrossEntropyLoss()(outputs, labels)  
    model.zero_grad()  
    loss.backward()  
    return torch.clamp(images + epsilon * images.grad.sign(), 0, 1)
```

PGD:

python

```
def pgd_attack(model, images, labels, epsilon=0.03, alpha=0.01, steps=3):
    ori = images.clone().detach()
    images = ori + 0.001 * torch.randn_like(ori)
    for _ in range(steps):
        images.requires_grad = True
        outputs = model(images)
        loss = nn.CrossEntropyLoss()(outputs, labels)
        model.zero_grad()
        loss.backward()
        images = images + alpha * images.grad.sign()
        eta = torch.clamp(images - ori, min=-epsilon, max=epsilon)
        images = torch.clamp(ori + eta, min=0, max=1).detach()
    return images
```

3.3 Combined Loss

We computed all three losses and averaged them to stabilize training:

python

```
loss = (loss_clean + loss_fgsm + loss_pgd) / 3
```

This simple strategy yielded the best trade-off between clean and adversarial performance.

3.4 Training Parameters

Parameter	Value
Epochs	15
Batch Size	32
Optimizer	Adam
Learning Rate	0.001
Epsilon	0.03
PGD α	0.01
PGD Steps	3

4. Failed Experimental Approaches

We experimented with several regularization techniques that ultimately **reduced accuracy** below the 50% threshold:

4.1 Label Smoothing & Augmentation

python

```
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

transform = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(32, padding=4),
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
                           saturation=0.2, hue=0.1),
    transforms.ToTensor()
])
```

4.2 Delayed PGD + Weighted Loss

python

```
if epoch >= 5:
    loss = 0.5 * loss_clean + 0.25 * loss_fgsm + 0.25 * loss_pgd
else:
    loss = 0.6 * loss_clean + 0.4 * loss_fgsm
```

4.3 Scheduler

python

```
scheduler = torch.optim.lr_scheduler.StepLR(optimizer,
                                             step_size=10, gamma=0.5)
```

These experiments failed due to over-regularization and possible distribution shift introduced by aggressive augmentations.

5. Hyperparameter Tuning Results

We tried tuning:

- Learning rate
- PGD steps and step size
- Loss weighting schemes
- Data augmentations

Observed Results:

Attempt	Clean	FGSM	PGD
Initial accepted model	0.527	0.366	0.363
Tuned version (final)	0.513	0.399	0.381

Though adversarial performance improved, clean accuracy dropped marginally. These results suggested diminishing returns from tuning alone.

6. Critical Observation: Epoch Count

We discovered that **running for more than 20 epochs consistently led to evaluation failures**:

Epochs	Clean Accuracy	Eval Status
10	≥ 50%	Accepted
20	Around 50%	Accepted
30+	< 50%	Rejected
50	Low accuracy	Rejected

This suggests **overfitting to adversarial examples**, hurting generalization on clean inputs. Shorter training (10–20 epochs) yielded the best balance.

7. Evaluation Results

Final submission:

```
python
Submission response: {
  'clean_accuracy': 0.5133333333333333,
  'fgsm_accuracy': 0.399,
  'pgd_accuracy': 0.38066666666666665
}
```

These results reflect a robust, balanced model that satisfies the competition requirements.

8. Conclusion

Our most effective model was the **simplest one**:

- Standard ResNet-18
- Clean + FGSM + PGD input mixing
- Uniform loss averaging

- No label smoothing or scheduler

Through our experiments, we discovered that adversarial training rewards simplicity over complex regularization techniques. We also found that more epochs can harm robustness by causing overfitting to adversarial examples, which negatively impacts generalization on clean inputs. Additionally, label smoothing and heavy augmentation were detrimental to our model's performance, suggesting that these techniques may introduce unwanted distribution shifts in the adversarial training context.

GitHub Repository: https://github.com/abaadm1/TML25_A3_22/releases/tag/assignment-03-final